

Phase 1 DeepXDE Code Guide

Robot Joint Motor Housing Inverse PINN

안형준

1 문서 목적

이 문서는 `main_deepxde.py`의 코드 흐름을 설명하기 위한 해설 문서다.

역할	파일
비교용 PyTorch baseline	<code>main.py</code>
DeepXDE 버전 PINN 코드	<code>main_deepxde.py</code>
기존 코드 설명 문서	<code>code_guide/PHASE1_CODE_GUIDE.tex</code>

`main_deepxde.py`는 `main.py`를 지우거나 대체하지 않고, 같은 inverse PINN 문제를 DeepXDE 형식으로 따로 구현한 파일이다. 현재 버전은 `main.py`에서 값을 import하지 않고, 필요한 물리 상수와 helper 함수를 파일 안에 직접 정의한다.

2 한 줄 요약

DeepXDE 버전도 푸는 물리 문제는 같다.

$$\text{input} = (X, \tau), \quad \text{output} = \theta(X, \tau)$$

학습 대상 inverse parameter는 다음 두 개다.

$$\beta \rightarrow h, \quad S \rightarrow q_0$$

차이는 loss를 직접 PyTorch 함수로 하나씩 계산하지 않고, 다음 DeepXDE 객체에 맡긴다는 점이다.

DeepXDE 객체	역할
<code>TimePDE</code>	PDE residual과 학습점 관리
<code>IC</code>	초기조건 관리
<code>OperatorBC</code>	미분 경계조건 관리
<code>PointSetBC</code>	센서 데이터 조건 관리

3 실행 방법

현재 코드에서는 DeepXDE backend로 PyTorch를 사용한다.

```
python3 -m pip install deepxde
python3 main_deepxde.py --iterations 5000
```

짧게 구조만 확인하려면 다음처럼 실행할 수 있다.

```
python3 main_deepxde.py --iterations 10 --display-every 1
```

4 PyTorch baseline과 DeepXDE 코드의 대응

아래 표는 비교를 위한 대응 관계다. 실제 `main_deepxde.py`는 `main.py`를 import하지 않고 독립적으로 실행된다.

역할	<code>main.py</code>	<code>main_deepxde.py</code>
신경망	<code>MotorHousingPINN</code>	<code>dde.nn.FNN</code>
PDE residual	<code>pde_residual()</code>	<code>pde()</code>
PDE 점 샘플링	<code>sample_collocation_points()</code>	<code>dde.data.TimePDE</code>
초기조건	<code>initial_condition_loss()</code>	<code>dde.icbc.IC</code>
단열 경계조건	<code>boundary_condition_loss()</code>	<code>dde.icbc.OperatorBC</code>
센서 데이터 조건	<code>sensor_data_loss()</code>	<code>dde.icbc.PointSetBC</code>
학습 파라미터	<code>raw_beta, raw_s</code>	<code>dde.Variable</code>
학습 실행	아직 smoke test 중임	<code>model.compile(), model.train()</code>

5 물리 상수와 함수의 독립 정의

`main_deepxde.py`는 `main.py`에서 값을 import하지 않는다. DeepXDE 코드 파일 하나만 봐도 실행 조건을 이해할 수 있도록 형상값, 물성치, 무차원화 기준값, true parameter, helper 함수를 파일 안에 직접 정의한다.

코드 상단에는 `math`도 import한다. 이는 단면적 A 와 둘레 P 를 계산할 때 π 가 필요하기 때문이다.

```
import math
```

```
L = 0.15
R_OUTER = 0.04
R_INNER = 0.025
X0 = 0.30
SIGMA_X = 0.10
SENSOR_X = (0.25, 0.55, 0.85)

RHO = 2700.0
CP = 900.0
K = 167.0
T_INF = 25.0

DELTA_T_REF = 50.0
I_REF = 5.0
T_FINAL = 1200.0

H_TRUE = 40.0
Q0_TRUE = 8.0e4
```

입력값에서 자동 계산되는 값들도 같은 파일 안에서 계산한다.

```
ALPHA = K / (RHO * CP)
AREA = math.pi * (R_OUTER**2 - R_INNER**2)
PERIMETER = 2.0 * math.pi * R_OUTER
TAU_FINAL = ALPHA * T_FINAL / L**2
```

β_{true} 와 S_{true} 는 변환 함수가 정의된 뒤 같은 파일 안에서 계산한다.

```
BETA_TRUE = beta_from_physical_h(H_TRUE)
S_TRUE = s_from_physical_q0(Q0_TRUE)
```

다음 helper 함수들도 main_deepxde.py 안에 직접 들어 있다.

함수	역할
tau_from_time()	실제 시간 t 를 무차원 시간 τ 로 변환
time_from_tau()	무차원 시간 τ 를 실제 시간 t 로 변환
beta_from_physical_h()	h 를 β 로 변환
s_from_physical_q0()	q_0 를 S 로 변환
gaussian_heat_source()	$g(X)$ 계산
current_profile()	$i(\tau)$ 계산
physical_h_from_beta()	β 를 h 로 변환
physical_q0_from_s()	S 를 q_0 로 변환
synthetic_theta_no_conduction()	local ODE 기반 synthetic data 생성

이 방식은 main.py와 코드 중복이 생기지만, DeepXDE 버전이 독립적인 실행 파일이 된다는 장점이 있다.

5.1 파일 내부 구역

main_deepxde.py의 큰 구역은 다음 순서로 되어 있다.

구역	역할
1	사용자가 정하는 입력값
2	입력값에서 자동 계산되는 값
3	차원/무차원 변환 함수
4	heat source, current profile, synthetic data 함수
5	inverse parameter 양수 제약
6	DeepXDE용 sensor data 생성
7	command line option
8	DeepXDE model 구성
9	출력 helper
10	실행부

6 DeepXDE backend 설정

파일 상단에서 DeepXDE backend를 PyTorch로 지정한다.

```
os.environ.setdefault("DDE_BACKEND", "pytorch")
```

그래서 pde() 함수 안에서는 torch.Tensor와 PyTorch 연산을 그대로 사용할 수 있다.

7 양수 파라미터 처리

물리적으로 β , S , h , q_0 는 음수가 되면 이상하다. 따라서 main_deepxde.py에서도 raw parameter를 학습하고, softplus를 통과시켜 양수로 만든다.

```
def positive_parameter(raw_value):
    return F.softplus(raw_value)
```

$$\beta = \text{softplus}(\text{raw_beta})$$

$$S = \text{softplus}(\text{raw_s})$$

`raw_beta`와 `raw_s`는 DeepXDE에서 다음처럼 정의한다.

```
raw_beta = dde.Variable(0.0)
raw_s = dde.Variable(0.0)
```

초기값이 0이면 `softplus` 때문에 실제 초기 추정값은 다음과 같다.

$$\text{softplus}(0) \approx 0.693$$

8 Synthetic sensor data

센서 데이터는 `main_deepxde.py` 안에 직접 정의된 `synthetic_theta_no_conduction()` 으로 만든다. 다만 DeepXDE의 `PointSetBC`는 NumPy 배열을 받기 때문에 `torch.Tensor`를 NumPy로 변환한다.

Mac에서 PyTorch가 MPS backend를 쓰면 tensor가 `mps:0` 장치에 올라갈 수 있다. 이 상태에서는 바로 `.numpy()`를 호출할 수 없으므로, 먼저 `.detach().cpu()`로 계산 그래프에서 분리하고 CPU로 옮긴 뒤 NumPy 배열로 바꾼다.

```
def make_synthetic_sensor_data_numpy(n_time=41):
    ...
    theta_sensor = synthetic_theta_no_conduction(x_grid, tau_grid)
    return (
        xtau.detach().cpu().numpy().astype("float32"),
        theta_sensor.detach().cpu().numpy().astype("float32"),
    )
```

기본 센서 데이터 개수는 다음과 같다.

$$3 \text{ sensors} \times 41 \text{ time points} = 123 \text{ points}$$

주의할 점은 현재 synthetic data가 full PDE 해가 아니라는 것이다. 아직 전도항 θ_{XX} 가 없는 local ODE 기반 데이터다.

$$\frac{d\theta}{d\tau} = -\beta_{\text{true}}\theta + S_{\text{true}}i(\tau)^2g(X)$$

9 DeepXDE PDE residual

제안서 기준 무차원 PDE는 다음과 같다.

$$\frac{\partial\theta}{\partial\tau} = \frac{\partial^2\theta}{\partial X^2} - \beta\theta + Si(\tau)^2g(X)$$

residual 형태는 다음과 같다.

$$f_{\text{PDE}} = \theta_\tau - \theta_{XX} + \beta\theta - Si(\tau)^2g(X)$$

`main_deepxde.py`에서는 이 식이 `pde()` 함수에 들어간다.

```
def pde(xtau, theta):
    theta_tau = dde.grad.jacobian(theta, xtau, i=0, j=1)
    theta_xx = dde.grad.hessian(theta, xtau, component=0, i=0, j=0)

    x = xtau[:, 0:1]
    tau = xtau[:, 1:2]
```

```

beta = positive_parameter(raw_beta)
s_value = positive_parameter(raw_s)
i_tau = current_profile(tau)
g_x = gaussian_heat_source(x)

return theta_tau - theta_xx + beta * theta - s_value * (i_tau**2) * g_x

```

여기서 $x_{\text{tau}}[:, 0:1]$ 은 X , $x_{\text{tau}}[:, 1:2]$ 는 τ 다.

10 해석 영역

DeepXDE에서는 공간 영역과 시간 영역을 따로 만든 뒤 합친다.

```

geom = dde.geometry.Interval(0.0, 1.0)
time_domain = dde.geometry.TimeDomain(0.0, TAU_FINAL)
geomtime = dde.geometry.GeometryXTime(geom, time_domain)

```

즉 해석 영역은 다음과 같다.

$$0 \leq X \leq 1, \quad 0 \leq \tau \leq \tau_f$$

11 초기조건

초기조건은 다음과 같다.

$$\theta(X, 0) = 0$$

DeepXDE에서는 다음처럼 정의한다.

```
ic = dde.icbc.IC(geomtime, zero_numpy, on_initial)
```

`zero_numpy()` 는 초기 온도 $\theta = 0$ 을 반환하는 함수다.

12 단열 경계조건

제안서의 양끝 경계조건은 단열이다.

$$\frac{\partial \theta}{\partial X}(0, \tau) = 0$$

$$\frac{\partial \theta}{\partial X}(1, \tau) = 0$$

DeepXDE에서는 미분 조건을 `OperatorBC` 로 넣는다.

```

def adiabatic_bc(inputs, outputs, _):
    return dde.grad.jacobian(outputs, inputs, i=0, j=0)

left_bc = dde.icbc.OperatorBC(geomtime, adiabatic_bc, on_left_boundary)
right_bc = dde.icbc.OperatorBC(geomtime, adiabatic_bc, on_right_boundary)

```

여기서 $j=0$ 은 입력 (X, τ) 중 첫 번째 변수인 X 에 대한 미분이라는 뜻이다.

13 센서 데이터 조건

센서 데이터는 DeepXDE의 PointSetBC로 넣는다.

```
sensor_bc = dde.icbc.PointSetBC(sensor_xtau, sensor_theta, component=0)
```

이 조건은 센서 위치와 시간에서 신경망 예측값이 synthetic sensor data와 가까워지도록 만든다.

$$L_{\text{data}} = \frac{1}{N_s} \sum_i [\theta_\phi(X_i, \tau_i) - \theta_{\text{sensor},i}]^2$$

14 TimePDE 데이터 객체

DeepXDE의 TimePDE에 PDE, 초기조건, 경계조건, 센서 데이터 조건을 한 번에 넘긴다.

```
data = dde.data.TimePDE(  
    geomtime,  
    pde,  
    [ic, left_bc, right_bc, sensor_bc],  
    num_domain=args.num_domain,  
    num_boundary=args.num_boundary,  
    num_initial=args.num_initial,  
    num_test=args.num_test,  
    train_distribution="Hammersley",  
)
```

각 인자의 의미는 다음과 같다.

인자	의미
num_domain	PDE residual을 검사할 내부 점 개수
num_boundary	경계조건 점 개수
num_initial	초기조건 점 개수
num_test	테스트용 점 개수
train_distribution	학습점 샘플링 방식

15 신경망 구조

DeepXDE에서는 신경망을 다음처럼 만든다.

```
layer_sizes = [2] + [args.width] * args.depth + [1]  
net = dde.nn.FNN(layer_sizes, "tanh", "Glorot_normal")
```

기본값은 다음과 같다.

[2, 32, 32, 32, 1]

즉 입력은 X, τ 두 개이고, 출력은 θ 하나다.

16 학습 실행

main()에서는 먼저 Adam optimizer로 학습한다.

```
model.compile(
    "adam",
    lr=args.lr,
    external_trainable_variables=[raw_beta, raw_s],
)
model.train(iterations=args.iterations, display_every=args.display_every)
```

중요한 부분은 `external_trainable_variables`다. DeepXDE의 신경망 weight 뿐 아니라 β , S 에 해당하는 `raw_beta`, `raw_s`도 같이 학습시키라는 뜻이다.
 옵션으로 L-BFGS 후처리도 가능하다.

```
python3 main_deepxde.py --iterations 5000 --lbfgs
```

17 학습 결과 출력

학습이 끝나면 β , S 를 실제 물리 파라미터로 바꿔서 출력한다.

```
beta = positive_parameter(raw_beta).detach()
s_value = positive_parameter(raw_s).detach()
h_value = physical_h_from_beta(beta)
q0_value = physical_q0_from_s(s_value)
```

변환식은 `main_deepxde.py` 안에 직접 정의된 다음 두 함수가 사용하는 식이다.

```
physical_h_from_beta()
physical_q0_from_s()
```

$$h = \frac{\beta k A}{P L^2}$$

$$q_0 = \frac{S k \Delta T_{\text{ref}}}{I_{\text{ref}}^2 L^2}$$

18 명령행 인자

인자	기본값	의미
<code>--iterations</code>	5000	Adam 학습 반복 횟수
<code>--lr</code>	1.0×10^{-3}	learning rate
<code>--depth</code>	3	hidden layer 개수
<code>--width</code>	32	hidden layer neuron 개수
<code>--num-domain</code>	2560	PDE 내부 점 개수
<code>--num-boundary</code>	256	경계 점 개수
<code>--num-initial</code>	128	초기조건 점 개수
<code>--num-test</code>	1000	테스트 점 개수
<code>--sensor-time-points</code>	41	센서별 시간점 개수
<code>--display-every</code>	500	loss 출력 간격
<code>--seed</code>	0	random seed
<code>--lbfgs</code>	off	Adam 이후 L-BFGS 실행 여부

19 현재 DeepXDE 코드의 한계

현재 `main_deepxde.py`가 아직 하지 않는 것은 다음과 같다.

1. full PDE finite difference solver 기반 synthetic data 생성
2. loss weight tuning
3. temperature contour plot
4. sensor fitting plot
5. parameter convergence plot
6. 실제 실험 데이터 입력

가장 중요한 한계는 sensor data가 아직 전도항을 포함한 full PDE data가 아니라 local ODE 기반이라는 점이다.

θ_{XX} is not included in current synthetic data

따라서 현재 DeepXDE 코드는 최종 검증용이라기보다, DeepXDE로 inverse PINN 구조를 구현하고 학습 흐름을 확인하는 버전으로 보는 것이 좋다.

20 다음 구현 순서

추천 순서는 다음과 같다.

1. DeepXDE 설치 후 짧은 iteration으로 실행 확인
2. β, S 가 학습 중 변하는지 확인
3. loss weight를 조정해서 data loss와 PDE loss 균형 확인
4. finite difference solver로 full PDE synthetic data 생성
5. DeepXDE 결과와 PyTorch baseline 결과 비교
6. learned h, q_0 를 true value와 비교하는 plot 추가